Learning from small data: features, locality and directionality

Magdalena Markowska

Department of Linguistics, Stony Brook

Stony Brook University

iACS Institute for Advanced Computational Science

PhPhoNE 2025 @ MIT

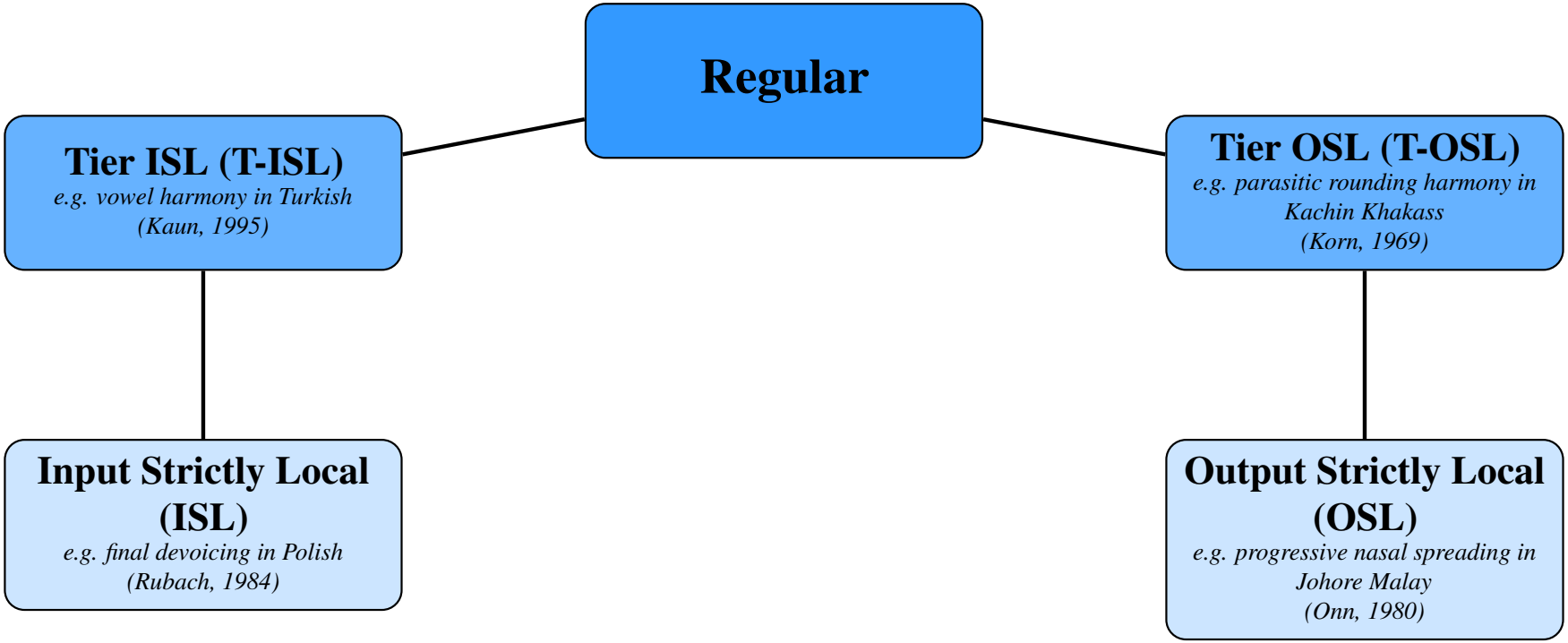
The subregular hypothesis

Subregular Phonology seeks to resolve the tension between theories of phonology that are *sufficiently expressive* to account for the cross-linguistic variation in phonological patterning and ones that are *sufficiently constrained* so that grammars can be learned from examples.

For processes these include the input and output strictly local maps and tier-based versions thereof (Chandlee and Heinz, 2018; Burness et al., 2021).

The subregular hypothesis

Subregular Phonology predicts that most phonological patterns fall within a subregion of regular languages. Multiple classes within this region have been identified (Heinz, 2018), for example:



THE SUBREGULAR HYPOTHESIS

Subregular Phonology predicts that most phonological patterns fall within a subregion of regular languages. Multiple classes within this region have been identified (Heinz, 2018), for example:

Regular

Tier ISL (T-ISL)
e.g. vowel harmony in Turkish
(Kaun, 1995)

Tier OSL (T-OSL)
e.g. parasitic rounding harmony in Kachin Khakass
(Korn, 1969)

Input Strictly Local (ISL)
e.g. final devoicing in Polish
(Rubach, 1984)

Output Strictly Local (OSL)
e.g. progressive nasal spreading in Johore Malay
(Onn, 1980)

Why does it matter?

Why the Subregular Hypothesis Matters

1. **Expressivity:**
   Captures both local and non-local phonological patterns
2. **Provable learnability results:**
   Subregular classes are mathematically well-defined, which allows for efficient algorithms that effectively generalize the observed patterns
3. **Computationally tractable:**
   Many patterns can be represented with *subsequential functions* (Heinz and Lai, 2013; Chandlee et al., 2014; Chandlee, 2017; Chandlee and Heinz, 2018) instantiated by *deterministic finite state transducers* (DFTs; Sakarovitch (2009)), making them suitable for various language technology applications

WHAT IS MY CONTRIBUTION?

Over the past two decades, theoretical work in subregular phonology has led to the development of various learning **algorithms** that can successfully identify morphophonological patterns. Here I focus on those that fall within the **ISL** class.

- ISLFLA (Chandlee et al., 2014): $O(n^2)$
- SOSFIA (Jardine et al., 2014): $O(n)$

What is my contribution?

Over the past two decades, theoretical work in subregular phonology has led to the development of various learning **algorithms** that can successfully identify morphophonological patterns. Here I focus on those that fall within the **ISL** class.

- ISLFLA (Chandlee et al., 2014): $O(n^2)$
- SOSFIA (Jardine et al., 2014): $O(n)$

Practical Limitation: large amounts of training data.

WHAT IS MY CONTRIBUTION?

Over the past two decades, theoretical work in subregular phonology has led to the development of various learning **algorithms** that can successfully identify morphophonological patterns. Here I focus on those that fall within the **ISL** class.

- ISLFLA (Chandlee et al., 2014): $O(n^2)$
- SOSFIA (Jardine et al., 2014): $O(n)$

Practical Limitation: large amounts of training data.

In this talk, I show:

- how to switch from **segment-based** to **feature-based** representations to significantly reduce the amount of data required for learning.
- why the **directionality** of string processing (left-to-right vs. right-to-left) also plays a crucial role in data efficiency.

Roadmap

1. k-ISL functions (Chandlee, 2014):
   - definition and representation
   - training data requirement that guarantees success –> why so large?
2. The role of features in learning phonological functions
3. Right-to-left vs. left-to-right processing
4. Case studies:
   - vowel nasalization in English
   - ə-epenthesis in Chukchi
   - opaque interaction of final devoicing and *o*-raising in Polish

k-ISL functions and k-ISL transducers

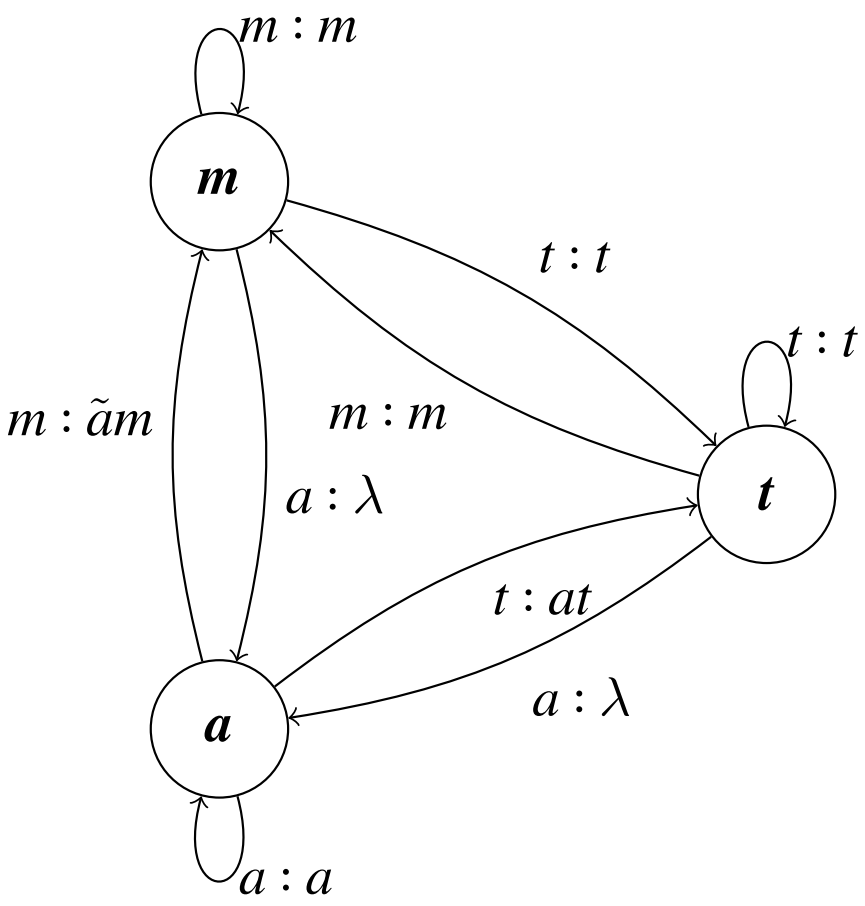
k-ISL functions can be instantiated with a *deterministic finite-state transducer (**DFT**)*, whose states and transitions are organized in such a way that the current state of the machine is always determined by the previous $k - 1$ symbols on the input.

Crucially, every k-ISL DFTs has **the same structure**.

- vowel nasalization in English ($VN \rightarrow \tilde{V}N$): $k = 2$
- final devoicing in Polish ($D\ltimes \rightarrow T\ltimes$): $k = 2$

Vowel nasalization as a 2-ISL process

$\Sigma = \{a, m, t\}$
$\Delta = \Sigma \cup \{\tilde{a}\}$

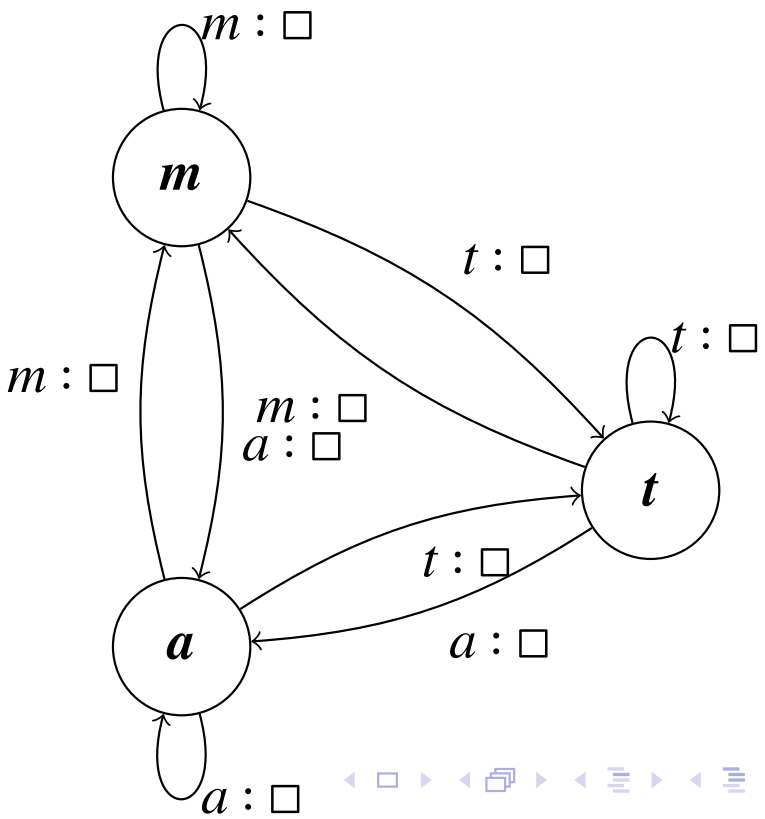


How do we learn such functions?

SOSFIA

Given a representative sample of input-output pairs and an output empty DFT for the relevant class, SOSFIA is **guaranteed** to calculate the right outputs, for every transition (i.e. learn the function).

...		...
/kæm/	→	[kæ̃m]
/kkm/	→	[kkm]
/mæm/	→	[mæ̃m]
/mæt/	→	[mæt]
...		...



Data requirements for SOSFIA

The size of k-ISL DFT depends directly on the window size k and the size of the input alphabet Σ.

The state space for such machines can be expresses as $|Q| = |\Sigma|^{k-1}$.

Because SOSFIA needs to see strings that correspond to paths for every transition, the **characteristic sample** S that guarantees success will be, roughly speaking, proportional to $|Q|$. Let's consider some numbers:

	$\lvert\Sigma\rvert$	$\lvert Q\rvert$	$\lvert S\rvert$
	4	4	84
$k = 2$	10	10	1,110
	50	50	127,550
	4	20	1,365
$k = 3$	10	110	111,100
	50	2550	318,877,550

Example

Transition: $q = (current_state, input, output, next_state)$
$\mathbf{f}_1$: k=2, $V \rightarrow \tilde{V}$ ___ M
$\mathbf{f}_2$: k=3, $V \rightarrow \tilde{V}$ ___ MM

	Transition:	**Required data**:
$\mathbf{f}_1$	(⋉, a, λ, a)	{aa–aa, am–ãm, at–at}
	(a, a, a, a)	{aaa–aaa, aat–aat, aam–aãm}
	(a, m, ãm, m)	{ama–ãma, amt–ãmt, amm–ãmm}
	(a, ⋊, a, ⋊)	{a–a}
	...	...
$\mathbf{f}_2$	(a, a, a, aa)	{aaa–aaa, aam–aãm, aaaa-aaaa, aaam–aaãm, ... }
	(aa, m, ãm, am)	{aama–aãma, aammm–aãmmm, aammt–aãmmt, ...}
	(am, t, t, mt)	{amta–ãmta, amtaa–ãmtaa, amtat–ãmtat ...}
	(mt, ⋊, λ, ⋊)	{mt–mt}
	...	...

Some assumptions and observations

- If no data available for a transition, the output is assumed to be *faithful* to the input.
- For *non-identical* transitions, the learner has to be exposed to very particular strings.
  - **The more non-identical transitions, the more data required.**
- The structure of a k-ISL DFT also directly affects the data size:
  - The bigger the k the bigger the k-ISL DFT.
  - The bigger the alphabet the bigger the k-ISL DFT.
  - **The bigger the k-ISL DFT, the more data the learner needs.**

Some assumptions and observations

- If no data available for a transition, the output is assumed to be *faithful* to the input.
- For *non-identical* transitions, the learner has to be exposed to very particular strings.
  - **The more non-identical transitions, the more data required.**
- The structure of a k-ISL DFT also directly affects the data size:
  - The bigger the k the bigger the k-ISL DFT.
  - The bigger the alphabet the bigger the k-ISL DFT.
  - **The bigger the k-ISL DFT, the more data the learner needs.**

The goal is then to:

I. Reduce the machine size

II. Reduce the amount of non-identical transitions

SOME ASSUMPTIONS AND OBSERVATIONS

- If no data available for a transition, the output is assumed to be *faithful* to the input.
- For *non-identical* transitions, the learner has to be exposed to very particular strings.
  - **The more non-identical transitions, the more data required.**
- The structure of a k-ISL DFT also directly affects the data size:
  - The bigger the k the bigger the k-ISL DFT.
  - The bigger the alphabet the bigger the k-ISL DFT.
  - **The bigger the k-ISL DFT, the more data the learner needs.**

The goal is then to:

I. Reduce the machine size → FEATURES

II. Reduce the non-identical transitions → DIRECTIONALITY

The main idea

Decompose the learning problem by generalizing over **features**.

- A learner predicts next feature values, as opposed to next symbol.

Let F be a finite set of features with binary or ternary values. If $f \in F$, ϕ_f is a homomorphism from Σ to the set $\{+, -, 0\}$.

- $\phi_{[\text{cor}]}(mat) = \begin{bmatrix} - & 0 & + \end{bmatrix}$
- $\phi_{[\text{cons}]}(mat) = \begin{bmatrix} + & - & + \end{bmatrix}$

If Φ is an ordered set of features, then Φ is a pointwise product of its homomorphisms.

- $\Phi_{\left[\begin{smallmatrix} \text{cor} \\ \text{cons} \end{smallmatrix}\right]}(mat) = \begin{bmatrix} - & 0 & + \\ + & - & + \end{bmatrix}$

Finally, if S is a sample of input/output pairs composed of segments, $S_{(F,f)}$ is sample for feature f.

- $S_{(F,f)} = \left\{ \left(\Phi_F(x),\ \Phi_f(y) \right) | (x, y) \in S \right\}$

Factoring by Feature (FxF)

Given a characteristic sample S, for each $f \in F$:

Step I. Project $S_{([f],f)}$.

Step II. Check if $S_{([f],f)}$ is *functional*.

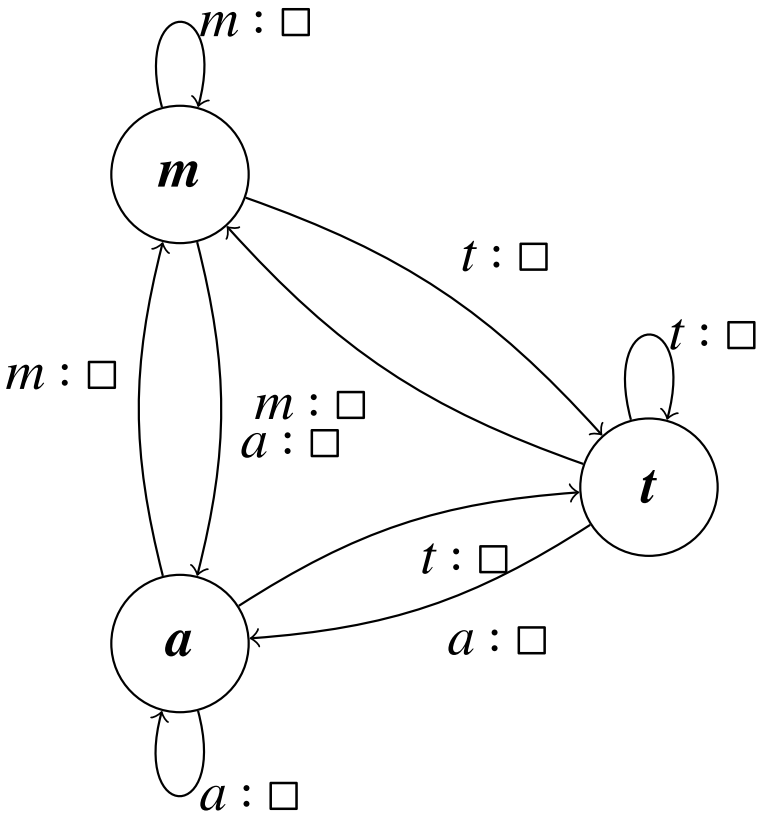
- A. If yes, given a $\tau_{\square}$, extract a k-ISL DFT_f and calculate the outputs with a learner (e.g. SOSFIA)
- B. If not, pick another $f' \in F$ and project $S_{([f,f'],f)}$; repeat until the new sample is functional.

An example from English

$k = 2$

$F = \{\text{cons, nasal, cor, lat,...}\}$

...		...
/kæm/	→	[kæ̃m]
/kkm/	→	[kkm]
/mæn/	→	[mæ̃m]
/mæt/	→	[mæt]
...		...



FEATURE CONSONANTAL

$$k = 2$$
$$f = [\textbf{consonantal}]$$

...		...
/+−+/	→	[+−+]
/+++/	→	[+++]
/+−−/	→	[+−−]
/−−−/	→	[−−−]
...		...

Feature consonantal

$$k = 2$$
$$f = [\textbf{consonantal}]$$

...		...
/+−+/	→	[+−+]
/+++/	→	[+++]
/+−−/	→	[+−−]
/−−−/	→	[−−−]
...		...

check_if_functional

Feature consonantal

$$k = 2$$
$$f = [\textbf{consonantal}]$$

...		...
/+−+/	→	[+−+]
/+++/	→	[+++]
/+−−/	→	[+−−]
/−−−/	→	[−−−]
...		...

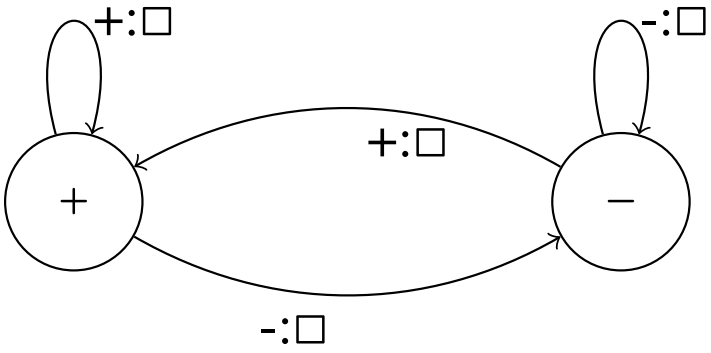
check_if_functional

True

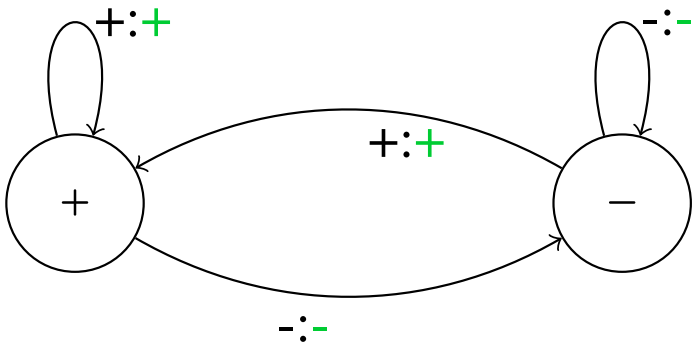
FEATURE CONSONANTAL

$$k = 2$$
$$f = [\textbf{consonantal}]$$

...		...
/+−+/	→	[+−+]
/+++/	→	[+++]
/+−−/	→	[+−−]
/−−−/	→	[−−−]
...		...



SOSFIA FxF's output for [consonantal]



FEATURE NASAL

$$k = 2$$
$$f = [\textbf{nasal}]$$

$$\begin{array}{ccc} \ldots & & \ldots \\ /--+/ & \rightarrow & [-++] \\ /--+/ & \rightarrow & [--+] \\ /+-+/ & \rightarrow & [+++] \\ /+-+/ & \rightarrow & [+-+] \\ \ldots & & \ldots \end{array}$$

Feature nasal

$k = 2$

f =[**nasal**]

$$
\begin{array}{ccc}
\ldots & & \ldots \\
/\,{-}{-}{+}\,/ & \rightarrow & [\,{-}{+}{+}\,] \\
/\,{-}{-}{+}\,/ & \rightarrow & [\,{-}{-}{+}\,] \\
/\,{+}{-}{+}\,/ & \rightarrow & [\,{+}{+}{+}\,] \\
/\,{+}{-}{+}\,/ & \rightarrow & [\,{+}{-}{+}\,] \\
\ldots & & \ldots
\end{array}
$$

check_if_functional

Feature nasal

$k = 2$

f =[**nasal**]

…		…
/ − − + /	→	[− + +]
/ − − + /	→	[− − +]
/ + − + /	→	[+ + +]
/ + − + /	→	[+ − +]
…		…

check_if_functional

False

Features nasal and consonantal

$k = 2$
$f, f' = [\textbf{nasal}, \text{consonantal}]$

$$
\begin{array}{ccc}
\ldots & & \ldots \\
/\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/ & \rightarrow & [-++] \\
/\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/ & \rightarrow & [--+] \\
/\begin{bmatrix}+\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/ & \rightarrow & [+++] \\
/\begin{bmatrix}+\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}-\\+\end{bmatrix}/ & \rightarrow & [--+] \\
\ldots & & \ldots
\end{array}
$$

Features nasal and consonantal

$$k = 2$$
$$f, f' = [\textbf{nasal}, \text{consonantal}]$$

...		...
$/\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/$	$\rightarrow$	$[-++]$
$/\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/$	$\rightarrow$	$[--+]$
$/\begin{bmatrix}+\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/$	$\rightarrow$	$[+++]$
$/\begin{bmatrix}+\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}-\\+\end{bmatrix}/$	$\rightarrow$	$[--+]$
...		...

check_if_functional

Features nasal and consonantal

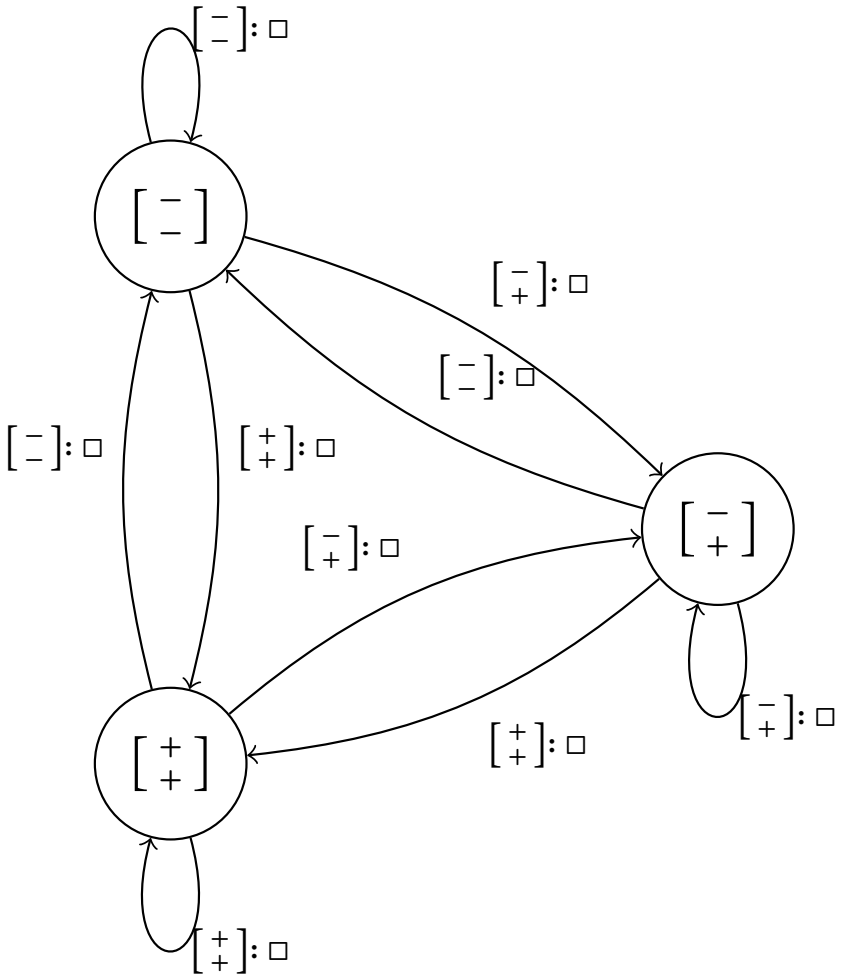
$k = 2$

$f, f' = [\textbf{nasal}, \text{consonantal}]$

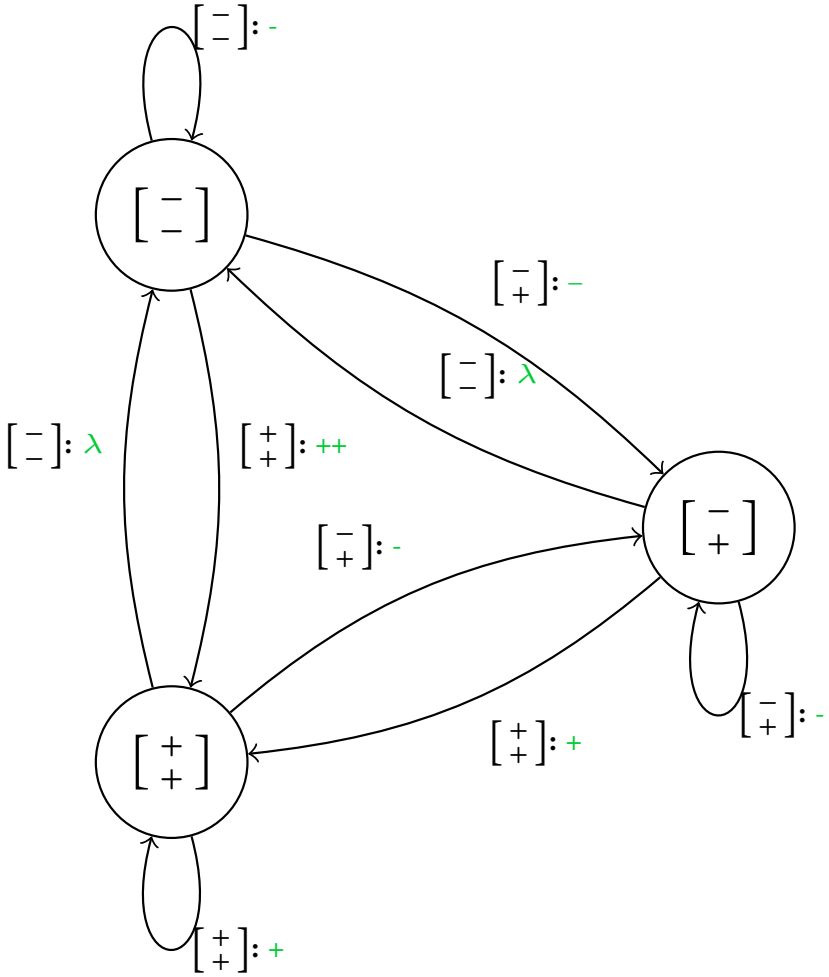
...		...
$/\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/$	$\rightarrow$	$[-++]$
$/\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}-\\+\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/$	$\rightarrow$	$[--+]$
$/\begin{bmatrix}+\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}+\\+\end{bmatrix}/$	$\rightarrow$	$[+++]$
$/\begin{bmatrix}+\\+\end{bmatrix}\begin{bmatrix}-\\-\end{bmatrix}\begin{bmatrix}-\\+\end{bmatrix}/$	$\rightarrow$	$[+--]$
...		...

check_if_functional True

OUTPUT EMPTY DFT$_{\left[\begin{smallmatrix}\textit{nasal}\\ \textit{cons}\end{smallmatrix}\right]}$



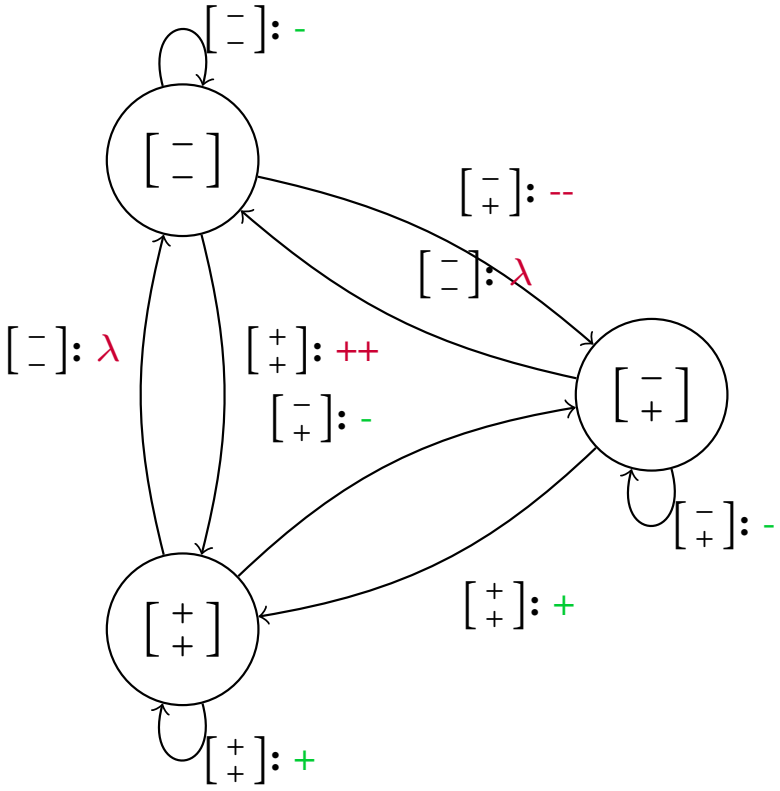
SOSFIA's output for $\begin{bmatrix} \textbf{\textit{nasal}} \\ \textit{cons} \end{bmatrix}$



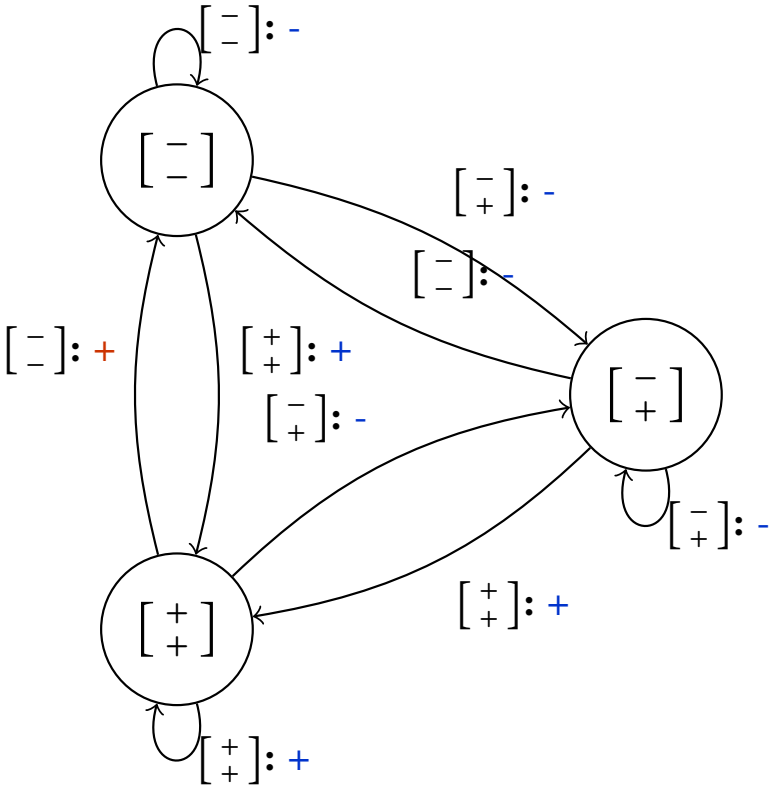
DIRECTIONALITY AND k-ISL FUNCTIONS

Whether a string is processed from left-to-right or right-to-left does not affect the **structure** of the machine (Chandlee, 2014).

left-to-right



right-to-left



Directionality and k-ISL functions

The comparison from before showed us that changing the direction of the string affects only the outputs: only the transitions leaving nasal consonant state ($\left[\begin{smallmatrix}+\\+\end{smallmatrix}\right]$) will be different from the inputs.

Essentially, **changing the direction can reduce** the number of non-identical transitions and thus the reduce the **data** needed.

Let's see how it all looks in practice!

Target k-ISL functions

- **Vowel Nasalization in English** (2-ISL; 22 features)
  [−cons] ⟶ [+nasal] / ___ [+cons, +nasal]

- **ə-epenthesis in Chukchi** (3-ISL; 22 features)
  ø⟶ ə / [+cons]___[+cons] ⋉

- **Final Devoicing and o-raising in Polish** (3-ISL; 17 features)
  [−sonorant] ⟶ [−voice] / ___ ⋉
  [-cons, +back, -low] ⟶ [+high] / ___ [+cons, +voice, -nasal]⋉

Alphabet and characteristic sample

- 2-ISL function:
  - $|\Sigma| = 26$
  - $|S| = 18,278$
- 3-ISL function:
  - $|\Sigma| = 7$
  - $|S| = 19,607$

Experiments

Each function is learned:

- over **segments**, tested for learnability
- separately for each **feature function** $f \in F$, also tested for success
- in both **left-to-right** and **right-to-left** processing directions

We also introduce a minimal sample search procedure: **AMβA**, which identifies a minimal subset of S sufficient for learning. We draw batches without replacement and test for success.

This yields two sample types:

- M: **segmental sample**, batch size = 50
- $M_{(F,f)}$: **featural sample** for feature f, batch size = 1
- We run **10 iterations** and report the average and standard deviation (SD)

Finally, we compute a **synthesis** of all individual featural samples to allow direct comparison with segmental performance.

English: Results

Feature f	$\lvert M_{(F,f)}\rvert$ Left-to-Right Avg.	Left-to-Right SD	Right-to-Left Avg.	Right-to-Left SD
$\left[\begin{smallmatrix}\textbf{nas}\\ \textit{cons}\end{smallmatrix}\right]$	31	3.88	29	10.10
$\left[\begin{smallmatrix}\textbf{nas}\\ \textit{lat}\end{smallmatrix}\right]$	125	17.25	–	–
[**round**]	8	2.27	6	3.57
[**cor**]	23	4.76	16	11.73
...	...	...	...	...
Synthesis (Avg.)	**189**	10.06	**137**	10.50
Segments (Avg. |M|)	**6,157**	650.21	**6,197**	563.62
|S|	18,278			

CHUKCHI: RESULTS

Feature f	$\lvert M_{(F,f)}\rvert$ Left-to-Right Avg.	Left-to-Right SD	Right-to-Left Avg.	Right-to-Left SD
[**round** *high*]	67	10.80	120	37.09
[**son** *low*]	237	27.90	81	20.87
[**cont** *tense*]	260	60.01	68	11.95
[**delrel** *long*]	1,174	92.80	360	146.29
[**nas** *son*]	230	42.89	97	18.54
[**voice** *cons*]	296	22.20	92	17.08
[**ant** *labdent*]	1,186	63.87	334	99.50
[**distr** *cor*]	1,155	87.70	254	103.92
[**strid** *dor*]	1,151	135.52	414	127.28
...	...	...	...	...
Synthesis (Avg.)	**2,533**	93.70	**1,401**	143.10
Segments (Avg. |M|)	**17,181**	28.87	–	–
|S|		19,607		

Polish: Results

Feature f	$\lvert M_{(F,f)}\rvert$ Left-to-Right Avg.	Left-to-Right SD	Right-to-Left Avg.	Right-to-Left SD
[**voice** *son*]	188	34.92	86	23.14
[**high** *voice* *son* *low*]	2,496	205.09	1,944	203.97
[**tense** *voice* *son* *round*]	2,570	176.08	1,943	193.74
[**ant**]	146	35.30	186	29.04
[**dor**]	38	7.22	30	16.01
...	...	...	...	...
Synthesis (Avg.)	**3,832**	139.17	**2,715**	202.39
Segments (Avg. |M|)	**15,191**	2063.50	–	–
|S|	19,607			

Polish: Bigger Alphabet

	$\lvert M_{(F,f)} \rvert$					
	Left-to-Right			**Right-to-Left**		
Feature f	$\Sigma = 8$	$\Sigma = 9$	$\Sigma = 10$	$\Sigma = 8$	$\Sigma = 9$	$\Sigma = 10$
$\begin{bmatrix} \textbf{voice} \\ son \end{bmatrix}$	215	239	179	76	126	–
$\begin{bmatrix} \textbf{tense} \\ voice \\ son \\ round \end{bmatrix}$	1,506	–	–	2,186	–	–
$\begin{bmatrix} \textbf{tense} \\ voice \\ nasal \\ round \end{bmatrix}$	–	2,827	3,007	–	3,982	–
[**ant**]	137	173	134	144	158	–
[**dor**]	35	29	26	35	41	–
...	...	...	...	...	...	...
Synthesis	**2,225**	**3,851**	**3,911**	**2,844**	**5,823**	–
Seg (Avg. |M|)	**30,344**	**52,161**	**79,540**	–	–	–
|S|	37,448	66,429	111,110	37,448	66,429	111,110

Conclusion

Key findings

- Learning phonological patterns is sensitive to how we represent the data — switching from segments to **features** drastically reduces data requirements.
- **Directionality matters**: in many cases, right-to-left processing leads to smaller and more efficient models.
- A sample search algorithm (**AMβA**,) successfully identifies a smaller sufficient sample. (We can definitely do better!)
- These results offer a roadmap for building interpretable, data-efficient learning models grounded in phonological structure.

Future Steps

1. Automatically learn the optimal k **value** for individual features
2. Evaluate the approach on more **naturalistic datasets**, including textbook-style data sets
3. Extend the analysis to other classes: **TISL, OSL, TOSL**
4. Design a **searchable model space** with controlled variables — k, features, directionality, tiers — to identify the most data-efficient configurations
5. Incorporate **noisy and incomplete data** to evaluate robustness
6. Integrate **learning lexicon**

THANK
YOU

Burness, P. A., K. J. McMullin, and J. Chandlee (2021). Long-distance phonological processes as tier-based strictly local functions. Glossa: a journal of general linguistics 6(1).

Chandlee, J. (2014). Strictly Local Phonological Processes. Ph. D. thesis, University of Delaware.

Chandlee, J. (2017). Computational locality in morphological maps. Morphology 27(4), 599–641.

Chandlee, J., R. Eyraud, and J. Heinz (2014). Learning strictly local subsequential functions. Transactions of the Association for Computational Linguistics 2, 491–504.

Chandlee, J. and J. Heinz (2018, January). Strict locality and phonological maps. Linguistic Inquiry 49(1), 23–60.

Heinz, J. (2018). The computational nature of phonological generalizations. In L. Hyman and F. Plank (Eds.), Phonological Typology, Phonetics and Phonology, Chapter 5, pp. 126–195. De Gruyter Mouton.

Heinz, J. and R. Lai (2013). Vowel harmony and subsequentiality. In A. Kornai and M. Kuhlmann (Eds.), Proceedings of the 13th Meeting on Mathematics of Language, Sofia, Bulgaria.

Jardine, A., J. Chandlee, R. Eyraud, and J. Heinz (2014, September). Very efficient learning of structured classes of subsequential functions from positive data. In A. Clark, M. Kanazawa, and R. Yoshinaka (Eds.), Proceedings of the Twelfth International Conference on Grammatical Inference (ICGI 2014), Volume 34, pp. 94–108. JMLR: Workshop and Conference Proceedings.

Sakarovitch, J. (2009). Elements of Automata Theory. Cambridge University Press. Translated by Reuben Thomas from the 2003 edition published by Vuibert, Paris.